

# Procesador de Paletas de Color

---

## Documentación Técnica

**Materia:** Procesamiento Digital de Imágenes

**Instituto:** IFTS 18

**Docente:** Juan Ignacio Bonini

**Integrantes:** Castaño Alma · Castelao Bravo María del Mar · Chirino Florencia

**Fecha:** Junio 2026

---

## 1. Descripción del Problema

El procesamiento de imágenes digitales implica, entre otras tareas, la extracción y manipulación del color como elemento fundamental de la información visual. En el campo del diseño, la publicidad y el análisis visual, la paleta de colores de una imagen define su identidad cromática: transmite emociones, genera contraste y determina la armonía estética.

El problema abordado en este proyecto consiste en desarrollar una herramienta capaz de:

1. **Extraer automáticamente** los colores dominantes de cualquier imagen digital.
2. **Generar variaciones cromáticas** basadas en relaciones teóricas del círculo cromático (armonías de color).
3. **Aplicar** la paleta seleccionada sobre la imagen original, produciendo una versión recolorada.
4. **Visualizar y comparar** los resultados de forma clara.

Este problema es relevante en contextos como la edición fotográfica, la generación de identidades visuales de marca y el análisis estético automatizado.

---

## 2. Enfoque y Diseño

La solución se diseñó como una aplicación modular de línea de comandos en Python, con una arquitectura orientada a objetos que separa responsabilidades en módulos bien definidos.

### 2.1 Principio de diseño

Cada módulo encapsula una responsabilidad única:

- **Análisis:** extracción de colores dominantes mediante clustering.
- **Generación:** producción de paletas alternativas usando teoría del color.
- **Edición:** aplicación de nuevos colores sobre la imagen.
- **Utilidades:** conversión y clasificación de valores de color.
- **Orquestación:** coordinación del flujo completo desde la entrada del usuario hasta los archivos de salida.

### 2.2 Flujo general

```
Imagen de entrada
↓
Carga y preprocesamiento (PIL)
↓
Clustering KMeans → Colores dominantes
↓
Generación de paletas (Complementaria / Análoga / Triádica)
↓
Selección del usuario
↓
Aplicación de paleta sobre imagen
↓
Guardado de resultados + imagen comparativa
```

## 3. Técnicas Utilizadas

### 3.1 Clustering con K-Means

Para extraer los colores dominantes de una imagen se utiliza el algoritmo **K-Means** de la biblioteca `scikit-learn`. Cada píxel de la imagen se trata como un punto en un

espacio tridimensional (R, G, B), y el algoritmo agrupa los píxeles en `k` clústeres, cuyos centros representan los colores dominantes.

**Parámetros clave:** - `n_clusters`: número de colores a extraer (1 a 8, definido por el usuario). - `random_state=42`: semilla fija para reproducibilidad. - La imagen se redimensiona a **200×200 píxeles** antes del clustering para reducir el costo computacional sin afectar la representatividad de la paleta.

### 3.2 Fusión de colores similares

Tras el clustering, se aplica un filtro de **distancia euclidiana** para eliminar colores casi idénticos. Si la distancia entre dos colores es menor a una tolerancia de 60 unidades, uno de ellos se descarta. Esto evita paletas redundantes.

```
distancia = np.linalg.norm(np.array(color) - np.array(existente))
if distancia < tolerancia:
    agregar = False
```

### 3.3 Teoría del color: armonías cromáticas

Las paletas alternativas se generan manipulando el canal de **tono (Hue)** en el espacio de color **HSV** (Hue-Saturation-Value):

| Tipo de paleta | Desplazamiento de tono |
|----------------|------------------------|
| Complementaria | +180°                  |
| Análoga        | -30° y +30°            |
| Triádica       | +120° y +240°          |

La conversión RGB ↔ HSV se realiza con el módulo estándar `colorsys` de Python.

### 3.4 Aplicación de paleta por umbral de proximidad

Para reemplazar los colores en la imagen, se utiliza una máscara numpy que identifica píxeles cuya distancia al color original es menor a **47 unidades** en cada canal:

```
mask = np.all(np.abs(img_array - original) < 47, axis=-1)
img_array[mask] = nuevo
```

### 3.5 Clasificación de color

Cada color se clasifica automáticamente según dos métricas:

- **Brillo:** promedio de los canales RGB. Clasificado como Oscuro (< 85), Medio (85–170) o Claro (> 170).
- **Temperatura:** comparación entre canal R y canal B. Si  $R > B \rightarrow$  Cálido; si  $B > R \rightarrow$  Frío; si  $R = B \rightarrow$  Neutro.

## 4. Implementación

### 4.1 Estructura de módulos

```
|— main.py           # Punto de entrada
|— paleta_app.py     # Orquestador principal (clase PaletaApp)
|— analizador_paleta.py # Carga, clustering y visualización
|— generador_paleta.py # Generación de paletas alternativas
|— editor_imagen.py  # Edición y aplicación de paleta
|— color.py          # Utilidades: HEX, brillo, temperatura
|— requirements.txt  # Dependencias del proyecto
```

### 4.2 Descripción de módulos

`color.py` — Clase `Color`

Encapsula la representación de un color RGB. Métodos principales:

| Método                                | Descripción                        |
|---------------------------------------|------------------------------------|
| <code>rgb_to_hex()</code>             | Convierte RGB a string hexadecimal |
| <code>calcular_brillo()</code>        | Promedio de canales R, G, B        |
| <code>clasificar_brillo()</code>      | Categoría: Oscuro / Medio / Claro  |
| <code>clasificar_temperatura()</code> | Categoría: Cálido / Frío / Neutro  |

`analizador_paleta.py` — Clase `AnalizadorPaleta`

Responsable de la carga y análisis de la imagen.

| Método                                              | Descripción                                                |
|-----------------------------------------------------|------------------------------------------------------------|
| <code>cargar_imagen()</code>                        | Abre y convierte la imagen a modo RGB                      |
| <code>extraer_colores()</code>                      | Aplica KMeans para obtener colores dominantes              |
| <code>fusionar_colores_similares(tolerancia)</code> | Elimina colores redundantes por distancia euclidiana       |
| <code>mostrar_resultados()</code>                   | Imprime en consola HEX, temperatura y brillo de cada color |
| <code>obtener_paleta_visual()</code>                | Genera imagen PNG con rectángulos de color y datos         |
| <code>obtener_paleta_original()</code>              | Devuelve la imagen original con recuadros superpuestos     |

`generador_paleta.py` — Clase `GeneradorPaletas`

Transforma la paleta base en variaciones cromáticas.

| Método                                   | Descripción                                                   |
|------------------------------------------|---------------------------------------------------------------|
| <code>_cambiar_tono(color, delta)</code> | Desplaza el tono en el espacio HSV                            |
| <code>generar_complementaria()</code>    | Aplica desplazamiento de 180°                                 |
| <code>generar_analogos()</code>          | Aplica desplazamientos de $\pm 30^\circ$                      |
| <code>generar_triadicos()</code>         | Aplica desplazamientos de 120° y 240°                         |
| <code>generar_paletas()</code>           | Devuelve diccionario con las tres paletas y sus descripciones |

`editor_imagen.py` — Clase `EditorImagen`

Operaciones de transformación sobre la imagen.

| Método                              | Descripción                                               |
|-------------------------------------|-----------------------------------------------------------|
| <code>cambiar_brillo(factor)</code> | Ajusta el brillo con <code>ImageEnhance.Brightness</code> |

| Método                                          | Descripción                                              |
|-------------------------------------------------|----------------------------------------------------------|
| <code>cambiar_saturacion(factor)</code>         | Ajusta la saturación con <code>ImageEnhance.Color</code> |
| <code>cambiar_tamano(ancho, alto)</code>        | Redimensiona la imagen                                   |
| <code>aplicar_paleta(originales, nuevos)</code> | Sustituye colores por proximidad (umbral 47)             |

`paleta_app.py` — Clase `PaletaApp`

Orquesta el flujo completo: entrada del usuario → análisis → generación → selección → edición → guardado.

`main.py`

Punto de entrada mínimo que instancia `PaletaApp` y llama a `ejecutar()`.

### 4.3 Dependencias

| Biblioteca                | Versión | Uso                                   |
|---------------------------|---------|---------------------------------------|
| <code>Pillow</code>       | 12.2.0  | Carga, edición y guardado de imágenes |
| <code>numpy</code>        | 2.4.6   | Operaciones matriciales sobre píxeles |
| <code>scikit-learn</code> | 1.9.0   | Algoritmo K-Means                     |
| <code>scipy</code>        | 1.17.1  | Dependencia de scikit-learn           |
| <code>joblib</code>       | 1.5.3   | Paralelismo en scikit-learn           |

## 5. Resultados

La aplicación fue probada con dos imágenes de referencia:

- **ladrillitos.jpg**: fotografía de textura con tonos rojizos y terrosos.
- **psy.jpg**: imagen con colores más variados y saturados.

## 5.1 Salidas generadas

Para cada ejecución se producen los siguientes archivos:

**Carpeta `PaletaOriginal/`** - `PaletaOriginal_Ncolores_nombreimagen.jpg` : imagen visual de la paleta extraída (rectángulos de color con HEX, temperatura y brillo). -

`PaletaOriginalFoto_Ncolores_nombreimagen.jpg` : imagen original con recuadros semitransparentes superpuestos mostrando los colores dominantes.

**Carpeta `PaletasGuardadas/`** -

`PaletaGuardada_Complementaria_Ncolores_nombreimagen.jpg` -

`PaletaGuardada_Analogica_Ncolores_nombreimagen.jpg` -

`PaletaGuardada_Triadica_Ncolores_nombreimagen.jpg`

**Carpeta `ImagenResultado/`** - `NombrePaleta_Ncolores_nombreimagen.jpg` : imagen con la paleta aplicada. - `COMPARATIVA_NombrePaleta_Ncolores_nombreimagen.jpg` : imagen lado a lado (original vs. editada) con información de la paleta aplicada.

## 5.2 Ejemplos de resultados

Los archivos disponibles en el repositorio evidencian resultados con 3, 4 y 5 colores sobre ambas imágenes de prueba, con paletas Análoga y Complementaria. El efecto de recolorización es notable especialmente cuando los colores de la paleta difieren significativamente en tono de los colores originales.

# 6. Pruebas

## 6.1 Validación de entrada

La aplicación valida la entrada del usuario en dos puntos críticos:

- **Ruta de imagen:** se verifica con `os.path.isfile()` antes de continuar. Si el archivo no existe, se solicita nuevamente.
- **Cantidad de colores:** se valida que sea un entero entre 1 y 8. Valores fuera de rango o no numéricos muestran mensajes de error descriptivos y repiten la solicitud.

## 6.2 Casos de prueba funcionales

| Caso                                     | Resultado esperado | Estado |
|------------------------------------------|--------------------|--------|
| Imagen válida, 3 colores, paleta Análoga |                    |        |

| Caso                                                      | Resultado esperado                      | Estado          |
|-----------------------------------------------------------|-----------------------------------------|-----------------|
|                                                           | Archivos generados en las tres carpetas | ✓<br>Verificado |
| Imagen válida, 4 colores, paleta Análoga                  | Archivos generados en las tres carpetas | ✓<br>Verificado |
| Imagen válida, 5 colores, paleta Complementaria           | Archivos generados en las tres carpetas | ✓<br>Verificado |
| Fuente de tipografía <code>arial.ttf</code> no disponible | Fallback a fuente por defecto de Pillow | ✓<br>Manejado   |
| Colores con alto solapamiento tras clustering             | Fusión automática reduce la paleta      | ✓<br>Verificado |

### 6.3 Casos de borde

- **Imágenes pequeñas:** el método `obtener_paleta_original()` recalcula proporcionalmente el tamaño de los recuadros para que sean visibles sin exceder la imagen.
- **Paletas con muchos colores similares:** `fusionar_colores_similares()` reduce la paleta efectiva, lo que puede resultar en menos colores de los solicitados.
- **Nombres de archivos con caracteres especiales o acentos:**  
`_normalizar_texto()` elimina acentos y caracteres no ASCII usando `unicodedata.normalize`.

## 7. Posibles Mejoras

### 7.1 Interfaz gráfica

La aplicación actualmente opera en línea de comandos. Una interfaz gráfica (por ejemplo con `tkinter` o una aplicación web con `Streamlit` o `Flask`) permitiría previsualizar los resultados en tiempo real, seleccionar la paleta visualmente y ajustar parámetros de forma interactiva.



## 7.2 Mejora del algoritmo de recolorización

El método actual de recolorización ( `aplicar_paleta` ) reemplaza píxeles por proximidad absoluta con un umbral fijo de 47 unidades. Esto puede generar bordes abruptos o colores no reemplazados en regiones con gradientes suaves. Alternativas a explorar:

- **Segmentación semántica** para reemplazar regiones completas en lugar de píxeles individuales.
- **Transferencia de estilo de color** usando el espacio Lab\* para una sustitución más natural.
- **Umbral adaptativo** calculado en función de la dispersión de cada clúster.

## 7.3 Más tipos de armonías

Se podría ampliar `GeneradorPaletas` con: - **Tetrádricas** (cuatro colores separados 90°). - **Split-complementarias** (un color y los dos adyacentes a su complementario). - **Monocromáticas** (variaciones de brillo y saturación del mismo tono).

## 7.4 Exportación a formatos de diseño

Añadir exportación de las paletas en formatos utilizados por herramientas profesionales: - **ASE** (Adobe Swatch Exchange) para Photoshop/Illustrator. - **JSON** para integración con aplicaciones web. - **CSS** con variables de color para desarrollo frontend.

## 7.5 Soporte para múltiples imágenes

Procesar un lote de imágenes en una sola ejecución, generando un informe consolidado de paletas para análisis comparativo.

## 7.6 Optimización del clustering

Incorporar métodos como el **Método del Codo** (Elbow Method) o el **índice de silueta** para sugerir automáticamente el número óptimo de colores sin que el usuario deba especificarlo.

---

## Apéndice: Instrucciones de Uso

### Instalación

```
pip install -r requirements.txt
```

### Ejecución

```
python main.py
```

La aplicación solicitará interactivamente:

1. La ruta de la imagen (ej: `ladrillitos.jpg`).
2. La cantidad de colores a extraer (entre 1 y 8).
3. La paleta a aplicar (1: Complementaria, 2: Análoga, 3: Triádica).

Los resultados se guardan automáticamente en `PaletaOriginal/`, `PaletasGuardadas/` e `ImagenResultado/`.